Python File Handling

```
import os
print(os.getcwd())
```

Text files

A text file stores information as characters encoded using a character encoding such as UTF-8.

Examples include:

- .txt — plain text
- .csv — comma-separated values
- .json — structured text data
- .log — log messages
- .py — Python source code

UTF-8 (Unicode Transformation Format – 8-bit) is a character encoding system used to represent text in computers

Opening and closing files

General syntax:

file = open(file_path, mode, encoding="utf-8")

- file_path: the path to the file.
- mode: specifies whether to read, write, append or perform another operation.
- encoding: specifies the character encoding in text mode.

Calling close() releases the file resource

Why should we close files?

An open file may hold operating-system resources

Failing to close it can cause resource leaks, delay writing of buffered data or interfere with other operations

```
file = open("employees.txt", "r", encoding="utf-8")
content = file.read()
print(content)
file.close()
```

Mode	Meaning	What it does
`"r"`	Read	Reads an existing file. Gives an error if the file does not exist.
`"w"`	Write	Creates a new file or **overwrites** an existing file.
`"a"`	Append	Adds new content at the end of a file. Creates the file if it does not exist.
`"x"`	Exclusive creation	Creates a new file. Gives an error if the file already exists.
`"r+"`	Read and write	Reads and writes an existing file without initially truncating it.
`"w+"`	Write and read	Reads and writes, but overwrites existing content.
`"a+"`	Append and read	Allows reading and appending to a file.

The with open() statement

The recommended approach is to use a context manager

```
with open("employees.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)
```

When execution leaves the with block, Python closes the file automatically, including when an exception occurs inside the block

Mode w: writing a file

```
with open("student.txt", "w", encoding="utf-8") as file:
    file.write("Anil\n")
    file.write("Sunita\n")
    file.write("Rahul\n")
```

Running this program again replaces the previous contents rather than adding to them

Append mode

```
with open("student.txt", "a", encoding="utf-8") as file:
    file.write("Meera\n")
```

Append mode is useful for adding records to an existing log or output file

Exclusive creation mode

```
with open("new_report.txt", "x", encoding="utf-8") as file:
    file.write("Monthly report")
```

The file is created if it does not exist. If it already exists, Python raises FileExistsError instead of overwriting it.

Using read()

Reads the entire file, or a specified number of characters in text mode

```
with open("students.txt", "r", encoding="utf-8") as file:
    content = file.read()

print(content)
```

Read only the first five characters:

```
with open("students.txt", "r", encoding="utf-8") as file:
    print(repr(file.read(5)))
```

Using readline()

- Reads one line at a time

```
with open("students.txt", "r", encoding="utf-8") as file:
    print(repr(file.readline()))
    print(repr(file.readline()))
```

Calling readline() again reads the next line. At the end of the file, it returns an empty string ("")

Using readlines()

- Reads all lines and returns them as a list

```
with open("students.txt", "r", encoding="utf-8") as file:
    lines = file.readlines()

print(lines)
```

Output:

```
['Anil\n', 'Sunita\n', 'Rahul\n', 'Meera\n']
```

Iterating through a file

- This is usually the best approach for processing large text files because it does not load the entire file into memory at once

```
with open("students.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

strip() removes leading and trailing whitespace, including newline characters.

Data engineering tip:

Prefer line-by-line iteration for large logs and newline-delimited records. Use read() when the file is small and the whole content is needed.

Writing files

Using write()

The write() method writes a string to a text file. It does not automatically add a newline.

```
with open("output.txt", "w", encoding="utf-8") as file:
    file.write("Python\n")
    file.write("Data Engineering\n")
    file.write("File Handling")
```

Using writelines()

The writelines() method writes multiple strings

It does not automatically insert newlines between them

```
lines = ["Apple\n", "Banana\n", "Orange\n"]

with open("fruits.txt", "w", encoding="utf-8") as file:
    file.writelines(lines)
```

If the strings do not contain newline characters, they will be written consecutively.

Writing data from a list

```
employees = ["E101", "E102", "E103"]


with open("employees.txt", "w", encoding="utf-8") as file:
    for employee in employees:
        file.write(employee + "\n")
```

Reading and writing CSV files

- CSV stands for Comma-Separated Values

- It is widely used for exchanging tabular data

Suppose employees.csv contains:

```
id,name,salary
101,Anil,35000
102,Sunita,45000
103,Rahul,40000
```

Reading a CSV file

```
import csv
with open("employees.csv", "r", newline="",
encoding="utf-8") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

Output:

['id', 'name', 'salary']
['101', 'Anil', '35000']
['102', 'Sunita', '45000']
['103', 'Rahul', '40000']

Each row is returned as a list of strings

newline=“” means Disable automatic newline translation

Suppose employees.csv contains:

Name,Address
Rahul,"Mysore
Karnataka"
Priya,Bengaluru

Notice that Rahul's address contains a newline inside quotation marks.

With newline="", Python's csv module correctly understands that Rahul's address spans two lines but belongs to one CSV record

Without it, newline translation can sometimes cause problems, especially with CSV files containing quoted fields with embedded newlines

Reading CSV using DictReader

```
import csv

with open("employees.csv", "r", newline="",
encoding="utf-8") as file:
    reader = csv.DictReader(file)

    for row in reader:
        print(row["name"], row["salary"])
```

DictReader uses the header row as dictionary keys, making the data easier to understand and process.

Writing a CSV file using DictWriter

```
import csv
employees = [
    {"id": 104, "name": "Meera", "salary": 50000},
    {"id": 105, "name": "Kiran", "salary": 42000}
]
with open("new_employees.csv", "w", newline="",
encoding="utf-8") as file:
    fieldnames = ["id", "name", "salary"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(employees)
```

Working with file positions: tell() and seek()

- Python maintains a current position within an open file
- tell() returns the current position
- seek() moves the position

```
with open("message.txt", "r", encoding="utf-8")
as file:
    print(file.tell())

    print(file.read(7))
    print(file.tell())

    file.seek(0)
    print(file.read(5))
```

- The first tell() returns 0 because reading has not started. After reading seven characters, the position is seven for this ASCII-only example
- seek(0) returns to the beginning
- For text files containing multibyte characters, text-stream positions are not necessarily the same as character counts or byte offsets

Current working directory

```
import os
print(os.getcwd())
```

Introduction to JSON

- JSON stands for JavaScript Object Notation
- It is a text-based format for representing structured data

Example JSON document

```
{
    "id": 101,
    "name": "Anil",
    "salary": 45000,
    "is_active": true,
    "skills": ["Python", "SQL", "ETL"]
}
```

JSON supports these main data types:

JSON type	Example	Python equivalent
Object	`{"name": "Anil"}`	`dict`
Array	`[10, 20, 30]`	`list`
String	`"Python"`	`str`
Number	`100` or `12.5`	`int` or `float`
Boolean	`true`, `false`	`True`, `False`
Null	`null`	`None`

Note the capitalization difference: JSON uses true, false, and null, whereas Python uses True, False, and None.

JSON parsing: JSON to Python

- Parsing means reading JSON text and converting it into Python objects

- The json.loads() function parses a JSON string

```
import json

json_data = '{"name": "Anil", "age": 25, "city": "Mangalore"}'

student = json.loads(json_data)

print(student)
print(type(student))
print(student["name"])
print(student["age"])
```

- json_data is a string containing valid JSON
- json.loads() converts the JSON object into a Python dictionary
- We can access values using dictionary keys

Reading a JSON file using json.load()

Suppose employee.json contains:

```
{
    "id": 101,
    "name": "Sunita",
    "department": "IT",
    "salary": 50000
}
```

```
import json

with open("employee.json", "r", encoding="utf-8")
as file:
    employee = json.load(file)

print(employee["name"])
print(employee["department"])
print(employee["salary"])
```

Remember the difference:

- json.loads() parses a JSON string
- json.load() parses JSON directly from a file object
- The letter s in loads is a useful reminder that the input is a string

JSON serialization: Python to JSON

- Serialization means converting a Python object into JSON-formatted text

Python provides:

- json.dumps() — converts a Python object into a JSON string
- json.dump() — writes a Python object directly to a file

Example: Convert a dictionary to a JSON string

```
import json
employee = {
    "id": 101,
    "name": "Anil",
    "salary": 45000,
    "active": True
}
json_string = json.dumps(employee)
print(json_string)
print(type(json_string))
```

Output:

{"id": 101, "name": "Anil", "salary": 45000, "active": true}

<class 'str’>

Notice that Python's True becomes JSON's true.

For readable JSON, use indent

```
import json
employee = {
    "id": 101,
    "name": "Anil",
    "skills": ["Python", "SQL"]
}
print(json.dumps(employee, indent=4))
```

Output:

```
{
    "id": 101,
    "name": "Anil",
    "skills": [
        "Python",
        "SQL"
    ]
}
```

Each key-value pair is indented by four spaces inside the opening and closing braces

Writing a JSON file using json.dump()

```
import json
employee = {
    "id": 101,
    "name": "Anil",
    "department": "IT",
    "salary": 45000,
    "skills": ["Python", "SQL"]
}

with open("employee.json", "w", encoding="utf-8") as file:
    json.dump(employee, file, indent=4)

print("JSON file created")
```

Working with nested JSON


```
{
    "order_id": "ORD1001",
    "customer": {
        "name": "Anil",
        "city": "Mangalore"
    },
    "items": [
        {
            "product": "Laptop",
            "quantity": 1,
            "price": 55000
        },
        {
            "product": "Mouse",
            "quantity": 2,
            "price": 700
        }
    ],
    "status": "confirmed"
}
```

```
import json

json_text = """
{
    "order_id": "ORD1001",
    "customer": {
        "name": "Anil",
        "city": "Mangalore"
    },
    "items": [
        {"product": "Laptop", "quantity": 1, "price": 55000},
        {"product": "Mouse", "quantity": 2, "price": 700}
    ]
}
"""
```

```
order = json.loads(json_text)
print(order["order_id"])
print(order["customer"]["name"])
print(order["customer"]["city"])
print(order["items"][0]["product"])
print(order["items"][1]["quantity"])
```

- The opening """ marks the beginning of the string, and the closing """ marks its end

- Everything between them is treated as a single string, including line breaks, spaces, braces, and quotation marks

- Calculate the total order value

```
total = 0

for item in order["items"]:
    total += item["quantity"] * item["price"]

print("Total order value:", total)
```

Handling JSON arrays

- A JSON array becomes a Python list

```
[
    {"id": 101, "name": "Anil", "salary": 45000},
    {"id": 102, "name": "Sunita", "salary": 50000},
    {"id": 103, "name": "Rahul", "salary": 40000}
]
```

Find employees earning at least ₹45,000

```
import json
json_text = """
[
    {"id": 101, "name": "Anil", "salary": 45000},
    {"id": 102, "name": "Sunita", "salary": 50000},
    {"id": 103, "name": "Rahul", "salary": 40000}
]
"""

employees = json.loads(json_text)

for employee in employees:
    if employee["salary"] >= 45000:
        print(employee["name"], employee["salary"])
```

Converting JSON to Python and Python to JSON

```
import json

# Python object
data = {
    "name": "Meera",
    "age": 28,
    "active": True,
    "skills": ["Python", "SQL"],
    "address": None
}

# Python -> JSON
json_text = json.dumps(data, indent=4)
print(json_text)
```

```
# JSON -> Python
restored_data = json.loads(json_text)
print(restored_data)
print(type(restored_data))
```

sort_keys

Sorts object keys alphabetically

```
import json

data = {"z": 10, "a": 20, "m": 30}

print(json.dumps(data, sort_keys=True))
```

ensure_ascii

- By default, ensure_ascii=True escapes non-ASCII characters in the JSON output
- Setting it to False makes Unicode text more readable

```
import json

data = {"city": "Mangaluru", "greeting": "ನಮಸ್ಕಾರ"}

print(json.dumps(data, ensure_ascii=False))
```

Thank You..